NGUYÊN LÝ HỆ ĐIỀU HÀNH

Bộ môn Mạng và An toàn thông tin

Chương 3

SỰ BẾ TẮC (DEADLOCK)

- Các tài nguyên của máy tính
- Khái niệm về sự bế tắc
- Giải thuật đà điểu
- Phát hiện và giải quyết bế tắc
- Phòng tránh bế tắc
- Ngăn chặn bế tắc

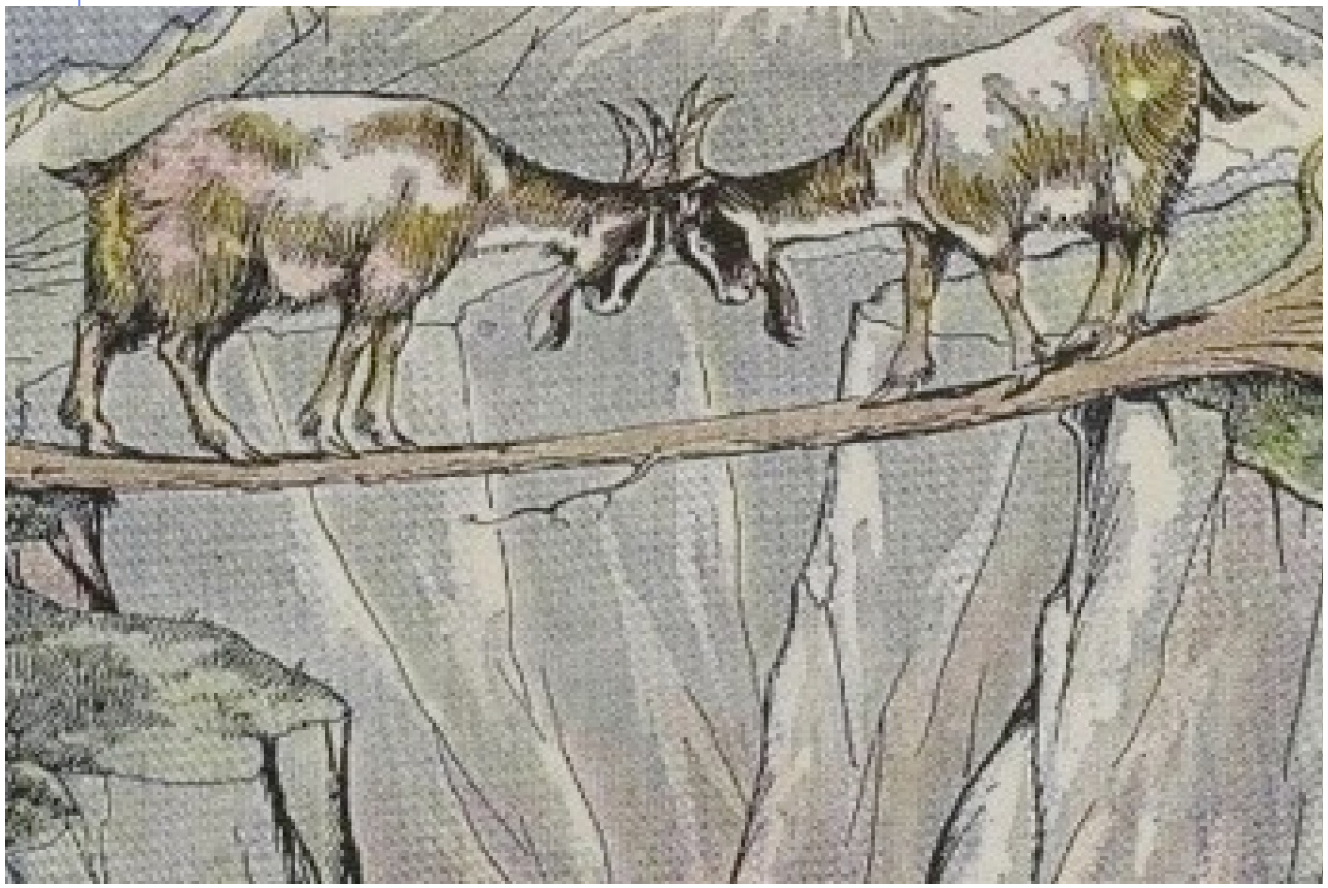
3.1 – Các tài nguyên

- Máy tính có thể là có nhiều loại tài nguyên như những thiết bị phần cứng, files, CSDLs,…
- Có 2 loại tài nguyên:
  - **Giải phóng được:** là loại tài nguyên có thể được giải phóng khỏi tiến trình sở hữu nó mà không gây ra hậu quả nào.
  - **Không giải phóng được:** là loại tài nguyên mà nếu lấy nó khỏi tiến trình đang sở hữu sẽ gây ra hậu quả nghiêm trọng

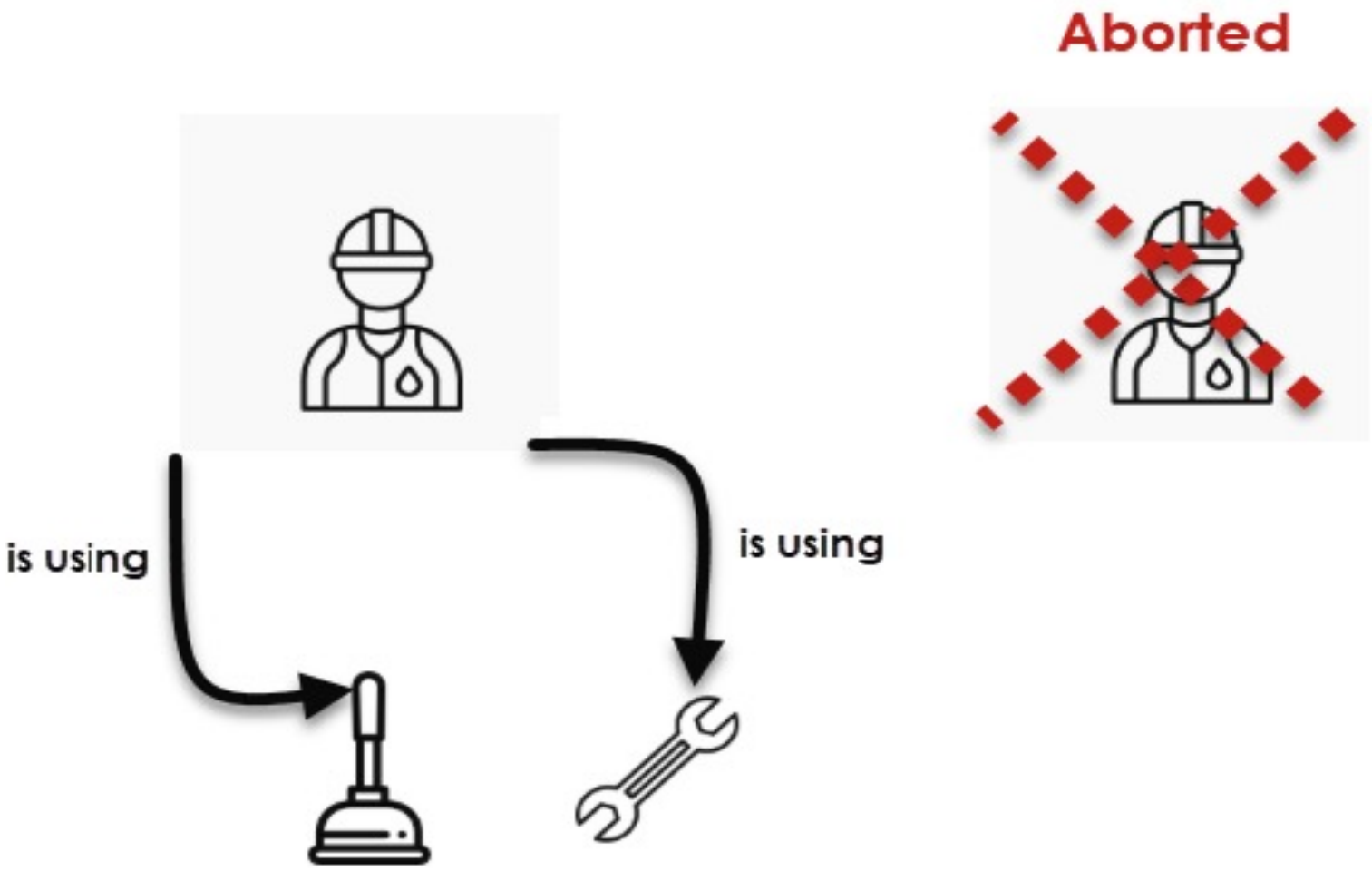
3.1 – Các tài nguyên

- Trình tự các sự kiện xảy ra trong quá trình/tiến trình sử dụng tài nguyên:
  - **Yêu cầu tài nguyên (Request):** Nếu không được hệ thống đáp ứng ngay thì tiến trình phải đợi cho đến khi có.
  - **Sử dụng** (Use): Tiến trình sử dụng tài nguyên
  - **Giải phóng (Release):** Tiến trình trả tài nguyên cho hệ thống

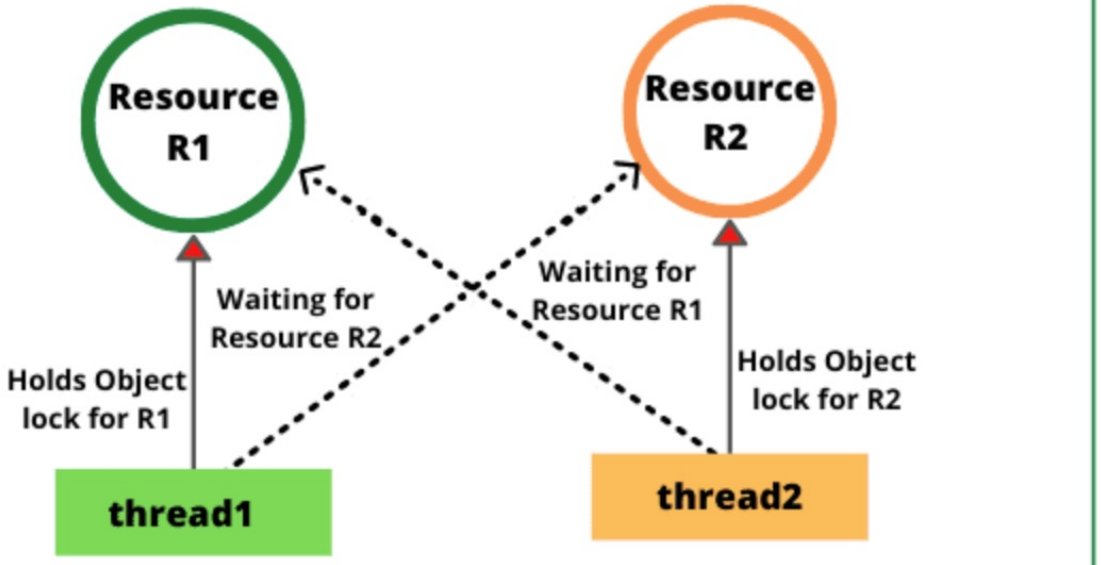
3.2 - Sự bế tắc



3.2 - Sự bế tắc

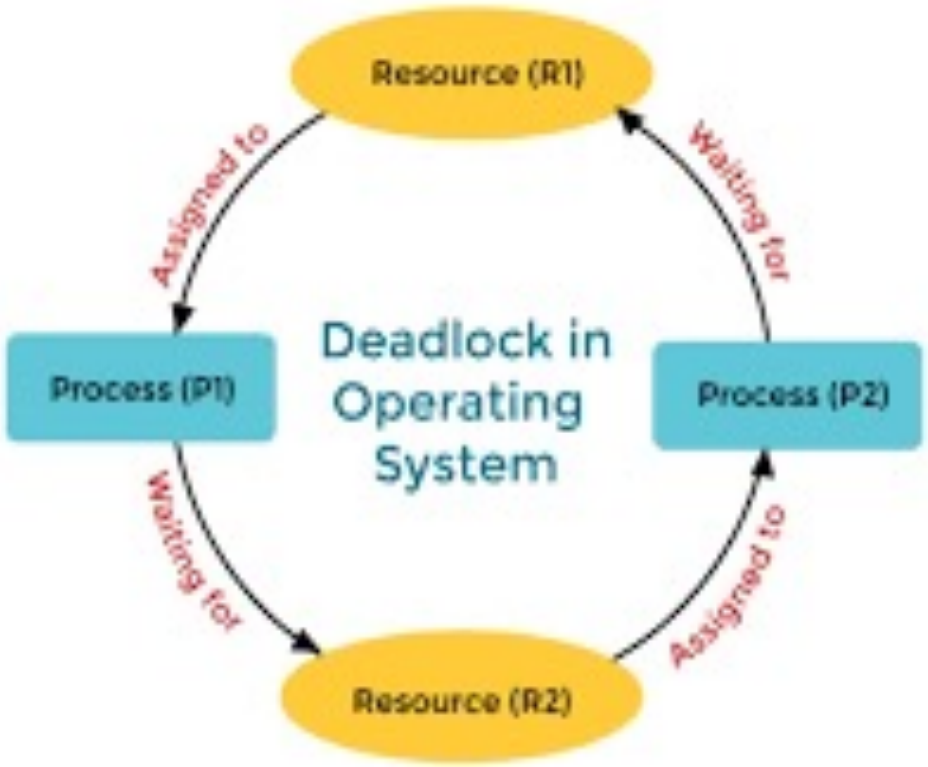


3.2 - Sự bế tắc



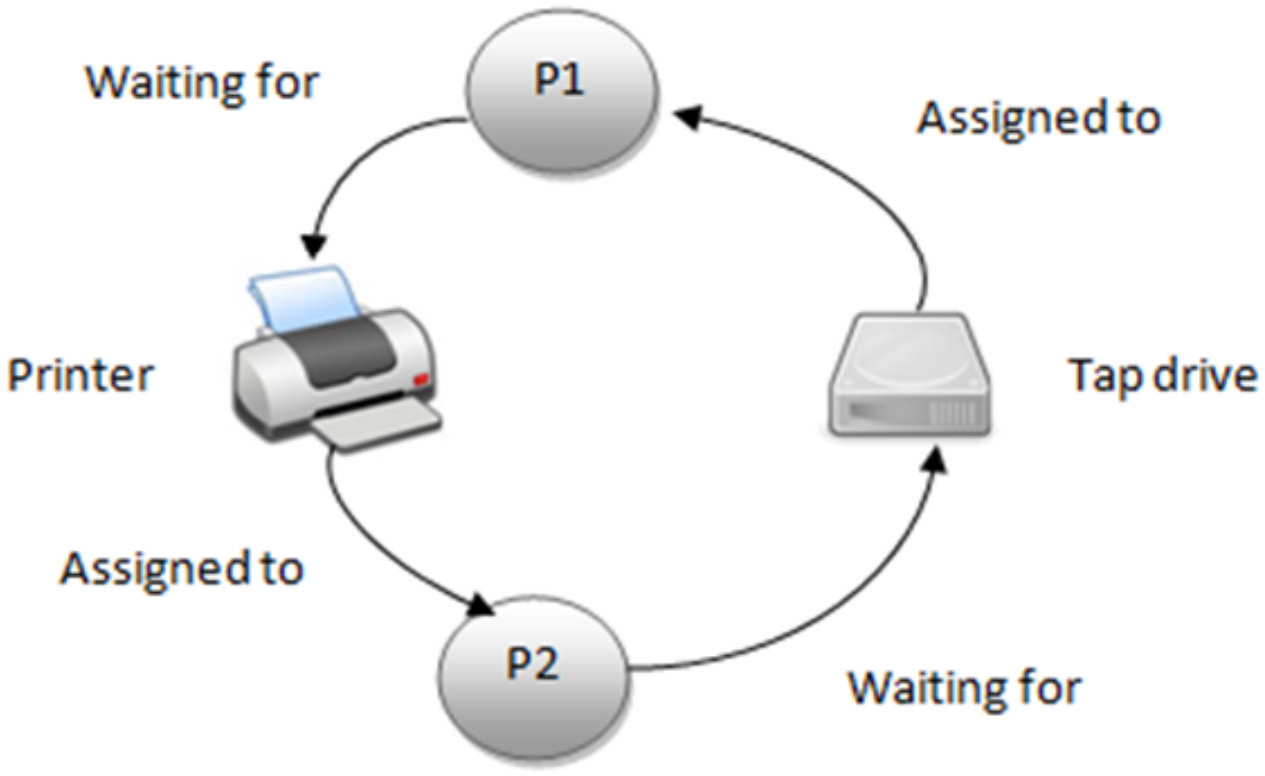
Here, both threads are waiting for each other to unlock resources R1 and R2, but thread2 cannot release lock for resource R2 until it gets hold of resource R1.

Fig: Example of Deadlock condition



3.2 - Sự bế tắc

- Khi có hai hoặc nhiều tiến trình, luồng tác động lẫn nhau, chúng có thể gây ra xung đột và không giải quyết được.
- Lúc này cả hai tiến trình đều bị dừng và sẽ phải chờ mãi mãi. Tình huống như vậy gọi là **sự bế tắc (deadlock)**.



Deadlock

3.2 - Sự bế tắc

- Sự bế tắc có thể định nghĩa như sau: ***Một tập hợp các tiến trình bị coi là bế tắc nếu mỗi tiến trình trong tập hợp phải chờ một sự kiện, mà sự kiện đó lại chỉ có thể do một tiến trình khác trong tập hợp tạo ra***

➔ Vì tất cả các tiến trình đều đang phải chờ, không tiến trình nào trong số chúng có thể tạo ra sự kiện cần thiết để đánh thức các thành viên trong tập hợp, nên chúng sẽ phải chờ mãi mãi

3.2 - Sự bế tắc

- Bế tắc là tình trạng:
  - Hai hay nhiều tiến trình cùng chờ đợi một sự kiện nào đó xảy ra
  - Nếu không có sự tác động gì từ bên ngoài thì sự chờ đợi đó là vô hạn
- Bế tắc là tình trạng không mong muốn vì khi xảy ra không tiến trình nào có thể kế thúc, tài nguyên hệ thống vẫn bị chiếm giữ.

3.2 - Sự bế tắc

- Nói chung, sự bế tắc thường liên quan tới các tài nguyên không giải phóng được. Cũng có những bế tắc liên quan tới tài nguyên giải phóng được, nhưng có thể giải quyết được chúng bằng cách phân phối lại tài nguyên từ tiến trình này sang tiến trình khác. Do đó ta sẽ tập trung vào các loại tài nguyên không thể giải phóng.

3.2 - Sự bế tắc (trong máy tính)

- Trong hệ thống máy tính có những loại tài nguyên chỉ cho phép một tiến trình được sử dụng ở một thời điểm
  - Ví dụ: Máy in, ổ băng từ, …
  - Việc hai tiến trình đồng thời sử dụng tài nguyên này sẽ gây ra bế tắc.
- Hệ điều hành phải có tính năng cho phép giải quyết bế tắc

Ví dụ: Xét tình huống sau đây

- Giả sử hai tiến trình A, B cùng muốn ghi tài liệu scan lên đĩa CD.
- Tiến trình A gửi yêu cầu muốn được cấp quyền sử dụng scanner và được chấp nhận.
- Tiến trình B gửi yêu cầu muốn được cấp quyền sử dụng ổ ghi CD và được chấp nhận.
- Sau đó, A muốn có ổ ghi CD, yêu cầu này bị từ chối do B đang sử dụng nó.
- Tiếp theo, B lại yêu cầu sử dụng scanner và cũng bị từ chối do A đang sử dụng scanner.
- ⇒ **Cả hai tiến trình đều bị dừng và phải chờ mãi mãi.**

  (Sự bế tắc)

3.2.1. Các điều kiện xảy ra bế tắc

Tình trạng bế tắc xảy ra khi bốn điều kiện sau đồng thời thỏa mãn (Coffman và các cộng sự đưa ra (1971)):

1) **Loại trừ tương hỗ/Độc quyền truy xuất (Mutual Exclusion):** Mỗi tài nguyên hoặc được sở hữu bởi một tiến trình duy nhất, hoặc đang rảnh rỗi. (1 tài nguyên chỉ phục vụ 1 tiến trình, tại 1 thời điểm chỉ có tối đa 1 tiến trình được quyền sử dung tài nguyên)

2) **Giữ và chờ (Hold and wait):** Các tiến trình đang nắm giữ tài nguyên được cấp trước đó có thể gửi yêu cầu đòi cấp tài nguyên mới (Phải có tiến trình đang nắm giữ tài nguyên và đồng thời lại chờ tài nguyên bị tiến trình khác chiếm giữ)

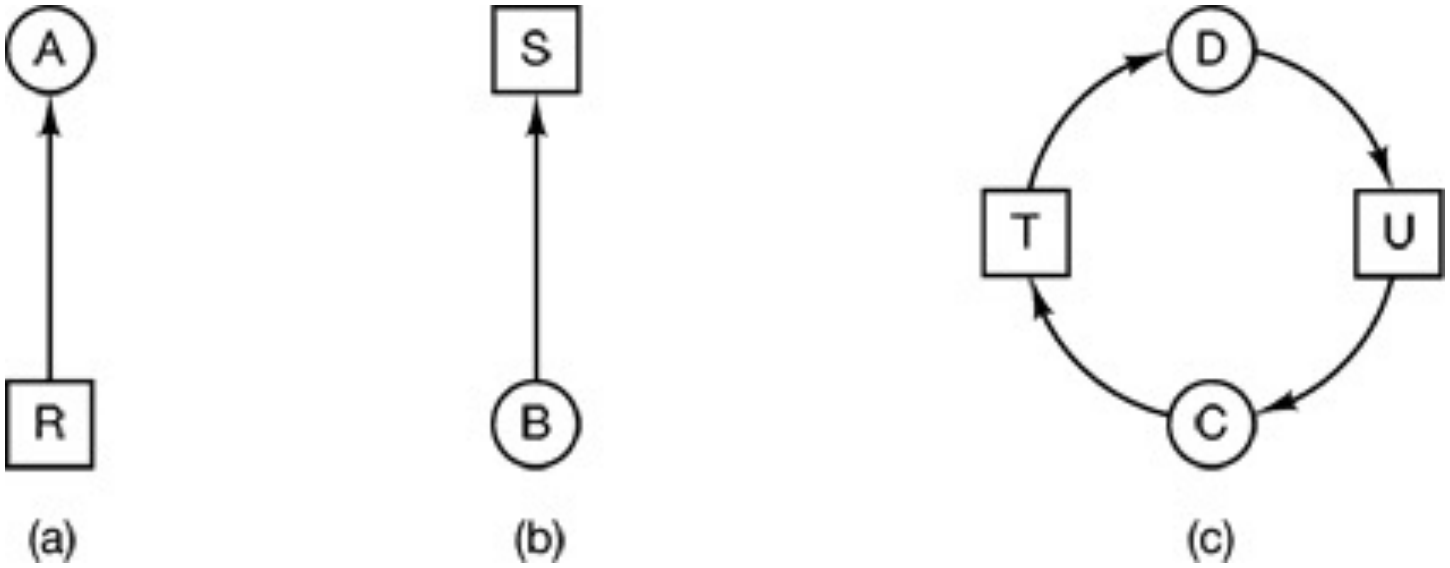
3) **Không giải phóng:** Tài nguyên do tiến trình giữ không thể phân phối lại cho tiến trình khác trừ khi tiến trình đang giữ tự nguyện giải phóng tài nguyên (Hệ thống không thể tước đoạt tài nguyên của tiến trình, trừ khi tiến trình chủ động giải phòng tài nguyên khi không cần thiết)

4) **Chờ đợi vòng tròn:** Phải có một hàng đợi vòng tròn gồm hai hoặc nhiều tiến trình, mỗi tiến trình lại đang chờ một tài nguyên được sở hữu bởi chính thành viên tiếp theo trong hàng đợi. {P0,P1,P3}, P0 chờ tài nguyên đang giữ bởi P1, P1 chờ P2, P2 chờ P3 và P3 lại chờ P0.

➔ ***Nếu không hội tụ đủ bốn điều kiện nói trên thì sẽ không có bế tắc!***

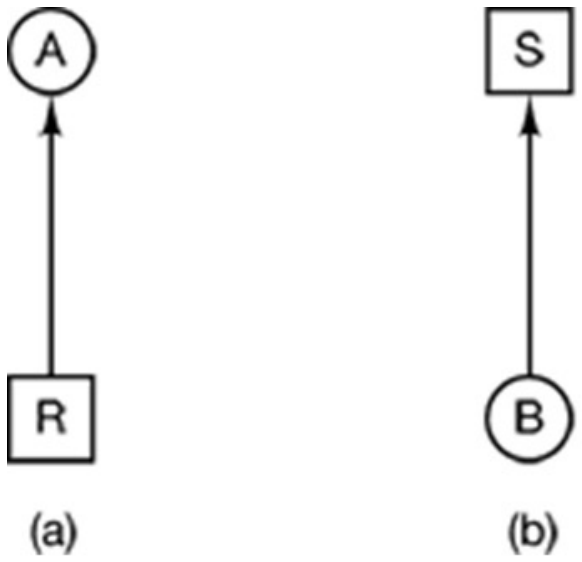
3.2.2. Mô hình hoá sự bế tắc

- Holt đã đưa ra cách mô hình hoá bốn điều kiện trên bằng các sơ đồ nút (năm 1972)



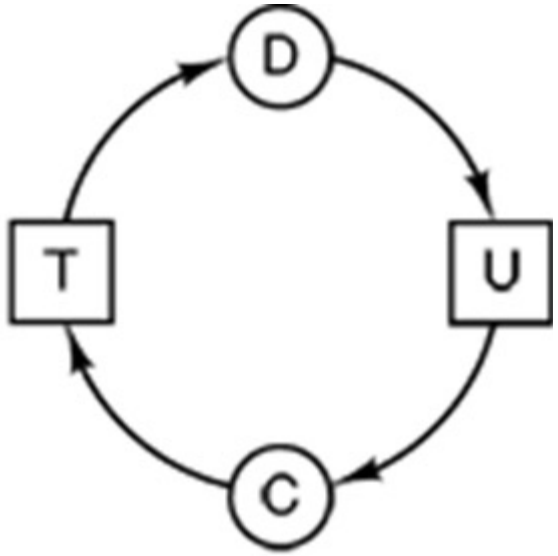
Sơ đồ phân phối tài nguyên: (a) Nắm giữ tài nguyên; (b) Yêu cầu tài nguyên; (c) Bế tắc xảy ra

3.2.2. Mô hình hoá sự bế tắc



- Nút tiến trình được khoanh tròn
- Nút tài nguyên được đóng khung vuông
- Mũi tên được nối từ nút tài nguyên tới nút tiến trình nghĩa là tài nguyên đó do tiến trình đó yêu cầu, và đã được phân cho tiến trình, tiến trình hiện đang sở hữu nó *(hình a)*
- Mũi tên được nối từ tiến trình tới tài nguyên, nghĩa là tiến trình đó hiện đang bị dừng để chờ nhận được tài nguyên đó *(hình b)*

3.2.2. Mô hình hoá sự bế tắc



(c)

Xét hình c:

- Tiến trình C đang chờ tài nguyên T
- Tài nguyên T lại do tiến trình D nắm giữ
- Tiến trình D không thể giải phóng T vì nó đang chờ tài nguyên U
- Tài nguyên U lại do C nằm giữ và không thể được giải phóng do C đang chờ tài nguyên T

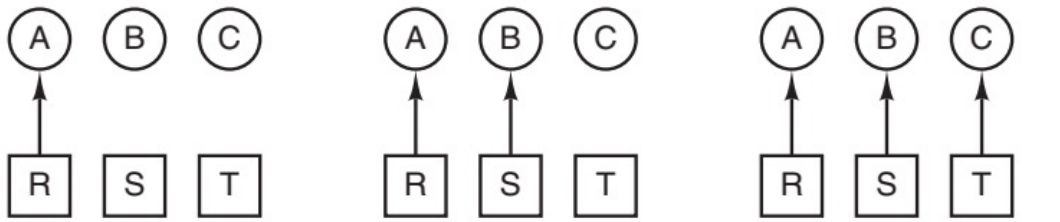
⇨ *Tạo thành một vòng khép kín, bế tắc xảy ra.*

Ví dụ: xảy ra bế tắc

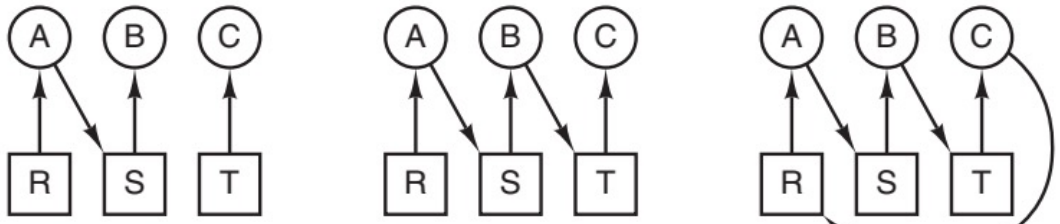
A	B	C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R
(a)	(b)	(c)

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R
   deadlock

(d)

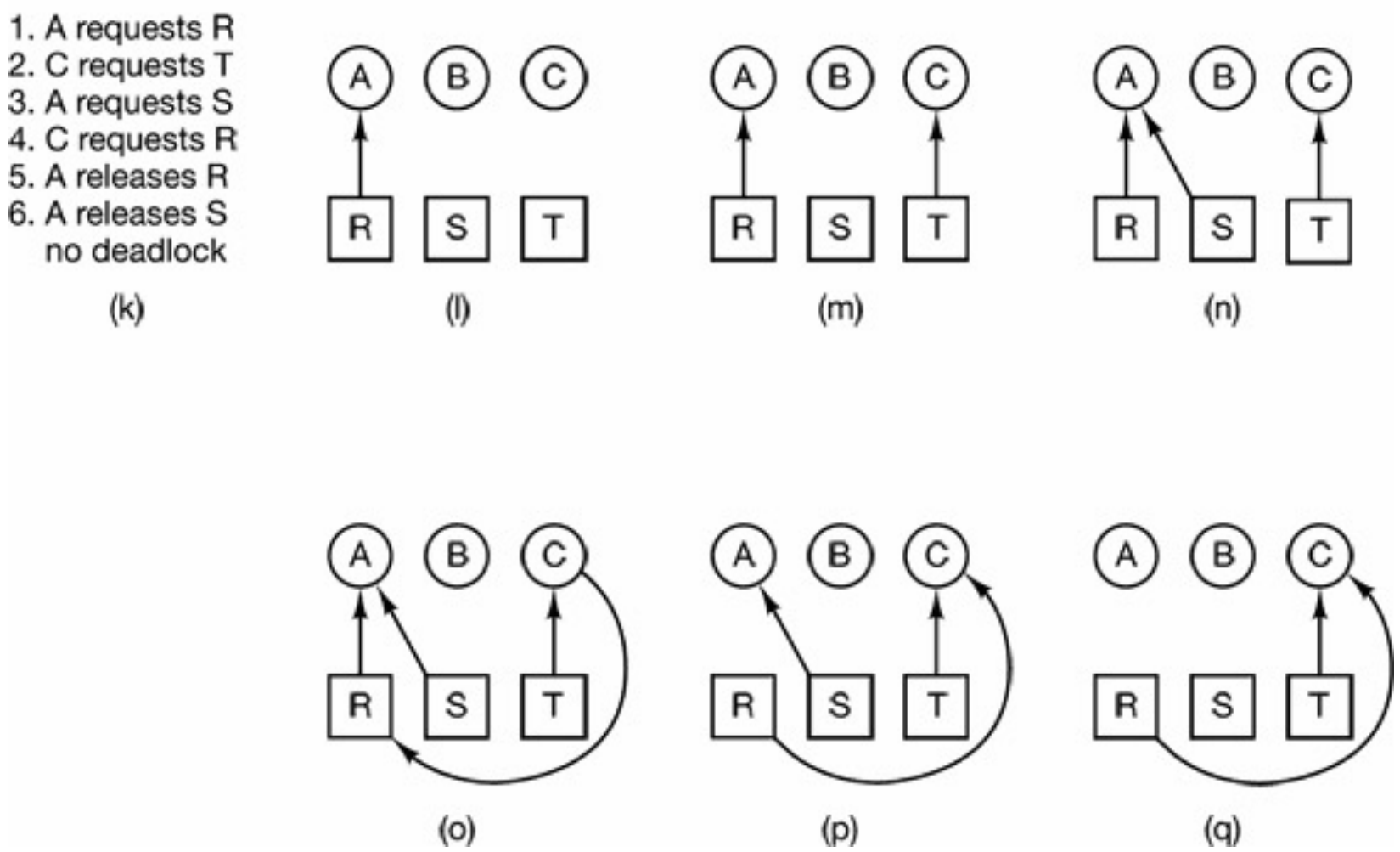


(e) (f) (g)



(h) (i) (j)

Cách phòng tránh: *Treo tiến trình B (hệ điều hành chỉ cho chạy A và C)*



Cách phòng tránh: *Treo tiến trình B (hệ điều hành chỉ cho chạy A và C)*

- Sau bước (q) có thể tiếp tục chạy tiến trình B và cấp cho nó tài nguyên S mà không gây ra bế tắc

➡ *Nếu việc thực hiện một yêu cầu của tiến trình có nguy cơ dẫn đến bế tắc, hệ điều hành có thể treo tiến trình mà không cần thực hiện yêu cầu đó cho tới khi thấy an toàn!*

3.3 – Xử lý bế tắc

Bốn chiến lược xử lý bế tắc

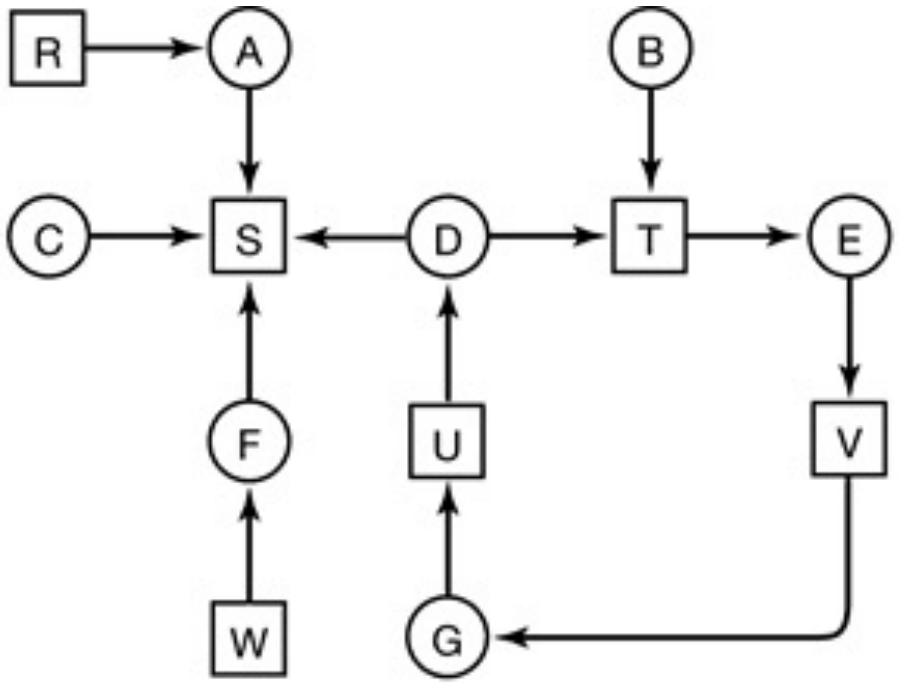
1) Bỏ qua tất cả các vấn đề (Giải thuật đà điểu)
2) Để cho các bế tắc xảy ra, phát hiện chúng và xử lý (*)
3) Chủ động phòng tránh bằng cách phân phối tài nguyên thật cẩn thận
4) Ngăn chặn, bằng cách loại bỏ sự tồn tại của một trong bốn điều kiện cần thiết gây ra bế tắc

Phát hiện bế tắc

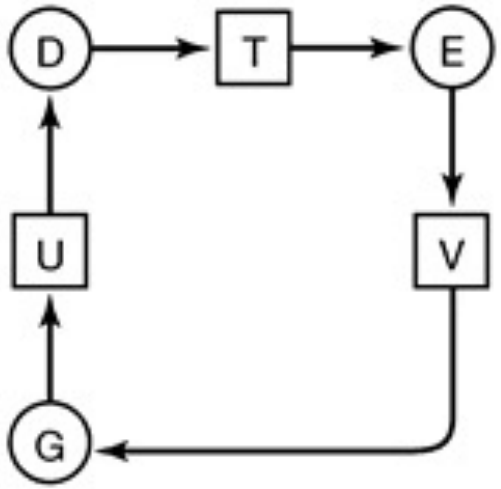
- **Ví dụ:** Xét hệ thống gồm có bảy tiến trình (từ A đến G), và có sáu tài nguyên (từ R đến W). Trạng thái của các tài nguyên và tiến trình như sau:
  - Tiến trình A đang nắm tài nguyên R và muốn có thêm S.
  - Tiến trình B không nắm tài nguyên nào và đang muốn có T.
  - Tiến trình C không nắm tài nguyên nào và đang muốn có S.
  - Tiến trình D đang sở hữu U và muốn có S và T.
  - Tiến trình E đang sở hữu T và muốn có V
  - Tiến trình F sở hữu W và muốn có S.
  - Tiến trình G sở hữu V và muốn có U.

"Hệ thống này có bị bế tắc không? và nếu có thì sẽ bao gồm những tiến trình nào?"

Giải: Vẽ sơ đồ phân phối tài nguyên



(a)



(b)

Kết luận:

- Các tiến trình D, E, G bị bế tắc vì tạo thành vòng kín
- Các tiến trình A, C, và F không bị bế tắc vì S có thể được phân phối cho chúng, khi dùng xong chúng có thể trả lại để tiến trình khác sử dụng

Bài tập

- 4 tiến trình: A, B, C, D; 4 tài nguyên: T, X, Y, Z
  - A sở hữu X và xin cấp Y
  - B xin cấp X và T
  - C sở hữu Y, xin cấp Z và T
  - D sở hữu Z, xin cấp Y

Hệ thống có bị bế tắc không? Nếu có thì những tiến trình nào bị bế tắc?

Giải thuật phát hiện vòng kín

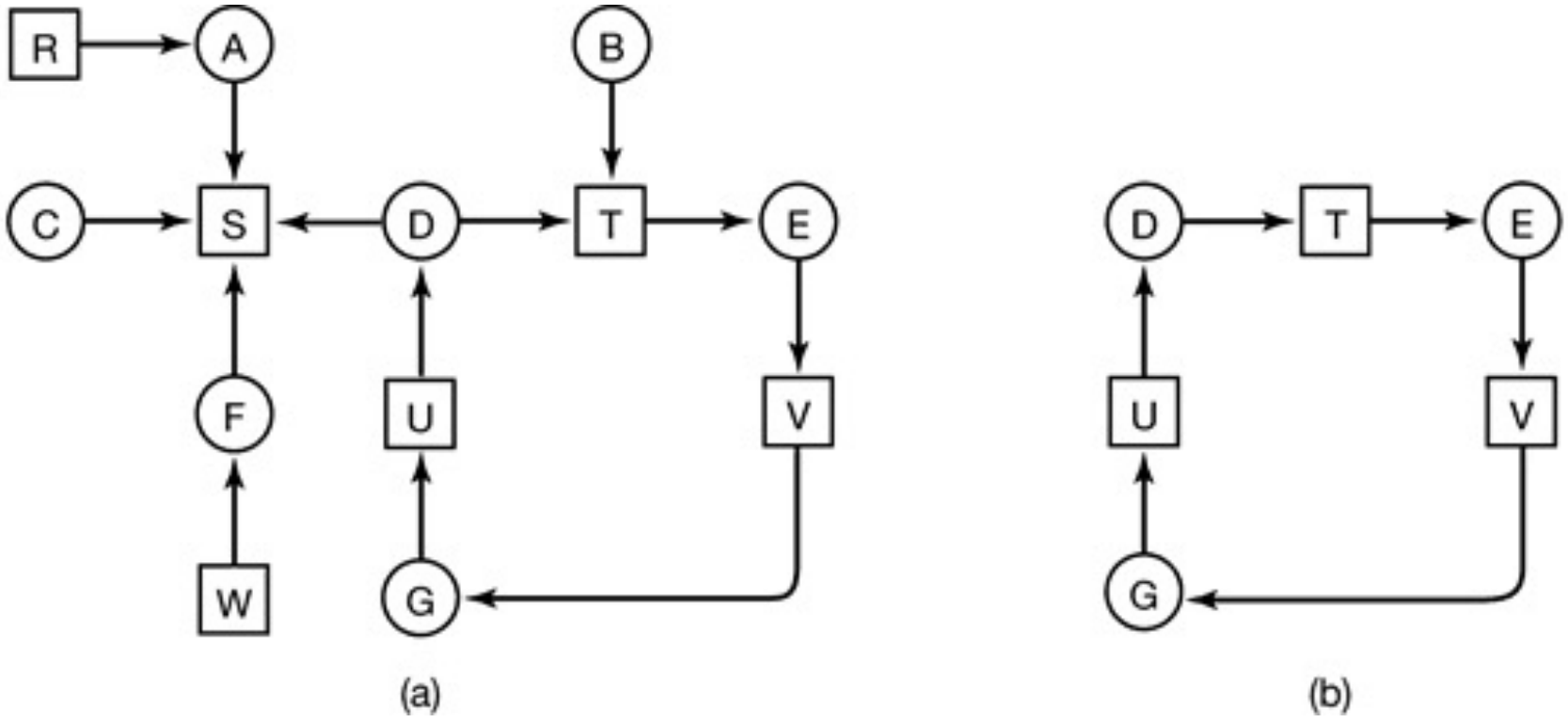
- Có rất nhiều giải thuật khác nhau để tự động phát hiện ra vòng kín, dưới đây ta xem xét một giải thuật đơn giản

Các bước của giải thuật

Giả sử có N nút trong sơ đồ. *Lần lượt thực hiện 5 bước sau đây cho từng nút*.

1. Khởi tạo L (là một danh sách rỗng) và đảm bảo rằng tất cả các mũi tên đều chưa bị đánh dấu.
2. Đặt nút hiện hành vào cuối danh sách L và kiểm tra xem liệu nút này có xuất hiện trong danh sách hai lần không: Nếu có thì tức là đã tạo thành vòng kín, giải thuật kết thúc; nếu không thì sang bước 3.
3. Kiểm tra nút hiện hành xem có mũi tên nào đi ra mà lại chưa được đánh dấu không. Nếu có thì chuyển sang bước 4, nếu không có thì chuyển sang bước 5.
4. Chọn ngẫu nhiên một mũi tên (trong số các mũi tên chưa được đánh dấu) rồi tiến hành đánh dấu nó. Đi theo hướng mũi tên đó để đến một nút mới, rồi quay trở lại bước 2.
5. Ta đã đi đến điểm cuối. Xoá bỏ nó và quay trở lại nút trước đó, tiếp tục áp dụng bước 2. Nếu nút này chính là nút ban đầu, tức là không phát hiện ra vòng kín nào theo nhánh này, quá trình kiểm tra hoàn thành.

Ví dụ minh họa

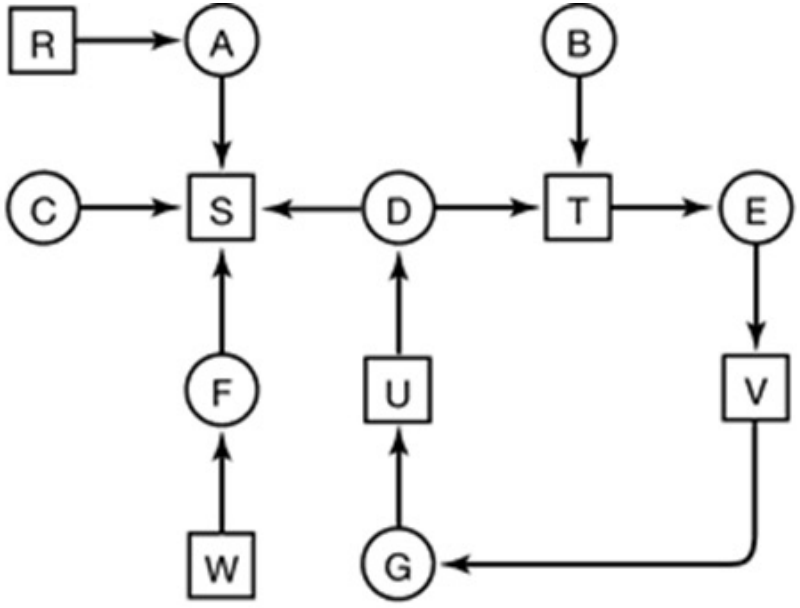


- Giả sử lần lượt kiểm tra các nút theo thứ tự: *R*, *A*, *B*, *C*, *S*, *D*, *T*, *E*, *F*…Nếu phát hiện ra một vòng kín thì giải thuật kết thúc.

Ví dụ minh họa

Bắt đầu từ nút R:

- Ban đầu L là một danh sách rỗng
- Đặt R vào danh sách rồi đi tới nút tiếp theo là A, thêm A vào danh sách. Như vậy *L* = [*R*, *A*].
- Từ A ta đi tới S, và *L* = [*R*, *A*, *S*].
- S không có mũi tên đi ra nào, nó là một điểm cuối, nên ta sẽ quay trở lại A. Do A không có mũi tên đi ra nào chưa được đánh dấu, nên ta sẽ quay lại R, hoàn thành việc kiểm tra với nút R.

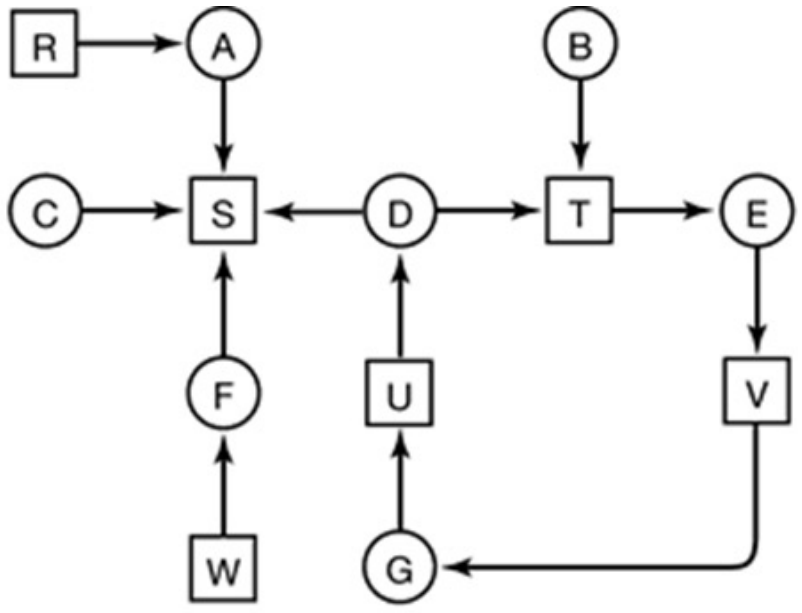


(a)

Ví dụ minh họa

Tiếp theo sẽ bắt đầu từ A:

- Danh sách L được làm rỗng
- Đặt A vào danh sách rồi đi tới nút tiếp theo là S, thêm S vào danh sách. Như vậy *L* = [*A*, *S*].
- S là một điểm cuối, nên ta sẽ quay trở lại A, hoàn thành việc kiểm tra với nút A.

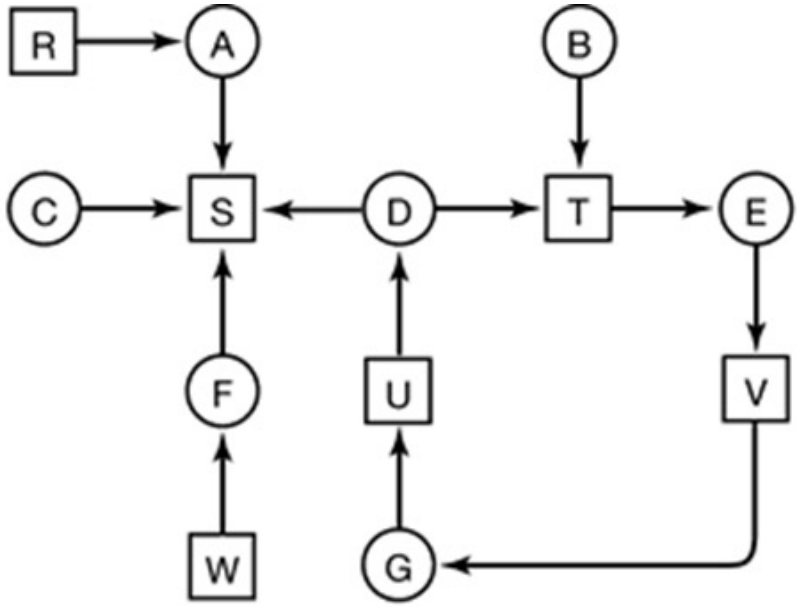


(a)

Ví dụ minh họa

Tiếp theo sẽ xuất phát từ B:

- Từ B ta đi theo các mũi tên hướng ra cho tới khi gặp D, lúc đó *L* = [*B*, *T*, *E*, *V*, *G*, *U*, *D*].
- Tại đây ta sẽ phải lựa chọn ngẫu nhiên nút tiếp theo. Nếu chọn nút S, đó là một điểm cuối, và ta sẽ quay lại D.
- Sau đó ta chọn được T và đặt vào danh sách: *L* = [*B*, *T*, *E*, *V*, *G*, *U*, *D*, *T*], tại điểm này ta đã phát hiện ra vòng lặp kín, giải thuật kết thúc!



(a)

Giải quyết bế tắc

Sau khi phát hiện bế tắc cần làm gì để hệ thống hoạt động trở lại?

1) Biện pháp giải phóng tài nguyên
2) Biện pháp quay trở lại
3) Biện pháp huỷ bỏ tiến trình

Biện pháp giải phóng tài nguyên

- Trong một số trường hợp, ta có thể tạm lấy tài nguyên của một tiến trình nào đó và giao cho tiến trình khác
- Biện pháp này phụ thuộc rất nhiều vào thuộc tính của tài nguyên đó và không dễ thực hiện.

Biện pháp quay trở lại

- Tiến trình đang sở hữu tài nguyên sẽ bị “reset” về thời điểm trước khi nó giành được tài nguyên. Tất cả các công việc đã làm sau thời điểm đó sẽ bị mất.
- Tài nguyên đó có thể được phân phối cho một tiến trình khác đang bế tắc.
- Khi tiến trình được khởi động trở lại, nếu nó muốn giành lấy tài nguyên, nó sẽ phải chờ cho tới khi tài nguyên đó sẵn sàng
- Cần có cơ chế lưu trữ trạng thái của tiến trình trong quá khứ, để có thể quay trở lại.

Biện pháp huỷ bỏ tiến trình

- Biện pháp thô bạo nhất, nhưng cũng đơn giản nhất để giải quyết bế tắc là huỷ bỏ một hoặc nhiều tiến trình.
- Có thể huỷ bỏ một tiến trình trong vòng kín. Nếu may mắn thì các tiến trình còn lại có thể tiếp tục hoạt động. Còn nếu chưa được thì có thể tiếp tục huỷ bỏ các tiến trình khác cho tới khi vòng kín bị phá vỡ.

Bài tập về nhà

- Các vấn đề *Phòng tránh bế tắc*, *Ngăn chặn bế tắc*... sinh viên tự nghiên cứu trong cuốn Modern Operating Systems.

Hết Chương 3